

# Anatomía de Merci Boilerplate

## v1.12 (Obsoleto)

**Estantería:** Boilerplate: Histórico

**Subtema:** Versiones 1.12.x

**mercedev.es** — 2026-05-12 | Epic 1 - Fase 10

La mayoría de plantillas web (boilerplates) actuales entregan un sitio funcional a costa de inyectar megabytes de librerías en el navegador, oscurecer la infraestructura y acumular deuda técnica desde el commit cero.

 **Aviso de obsolescencia:** Este documento relata la arquitectura original (v1.12). [Lee aquí la versión v1.15.1 actualizada.](#)

**Merci Boilerplate es lo opuesto.** Lo que comenzó como un proyecto de investigación para aplicar metodologías de Ciberseguridad y Automatización, terminó evolucionando hacia un ecosistema operacional completo.

El sistema surgió del rigor académico: se requería una arquitectura segura, sin frameworks opacos (Zero-Bloat), integrando sistemas Headless y aplicando metodologías Agile para mantener la deuda técnica a cero en cada fase. Todo guiado por especificaciones formales y documentado en una bitácora inmutable.

## 4 Verdades sobre este Boilerplate

---

### 1. No es una plantilla visual, es una infraestructura

Incluye más de 25 herramientas Python propias para automatizar tareas críticas: auditoría de código, generación SSG, sincronización con CMS, observabilidad de métricas e integración continua de IA.

### 2. Hybrid Stack de IA (Sin punto único de fallo)

La Inteligencia Artificial está orquestada de forma local y en la nube:

- **Ollama (qwen2.5-coder):** Ejecución local con latencia cero y privacidad absoluta de los datos.
- **Gemini Flash API:** Análisis semánticos complejos en la nube.
- **LiteLLM:** Actúa como proxy agnóstico.

- **Graceful degradation:** Si el proveedor Cloud se cae, el sistema local asume el control. Si ambos fallan, el pipeline no colapsa, simplemente se degrada a funciones estáticas.

### 3. Arquitectura 3 capas (Completamente aislada)

Las tres capas operan en paralelo sin contaminarse, unidas por Symlinks, Nginx y permisos estrictos:

- **Capa Estática:** HTML5 Semántico, SASS BEM y Vanilla JS (0 dependencias externas, 0 librerías NPM en *runtime*).
- **Capa Dinámica:** WordPress Headless para gestión de blogs, tiendas o bases de datos complejas.
- **Capa de Orquestación:** Agentes autónomos (Self-healing, Auto-documentación).

### 4. Gobernanza Operacional Reproducible

El repositorio incluye un *Deployment Playbook* paso a paso para servidores Ubuntu, SOPs (Standard Operating Procedures) de publicación dual y un *Checklist* de hardening. Todo lo necesario para operar en producción bajo estándares SRE.

## El Ecosistema de Agentes

---

El corazón del Boilerplate son sus 25+ agentes programados en Python puro, divididos en tres escuadrones:

### Auditoría & Seguridad (Escudo Activo)

- `merci-audit.py` : SAST (Static Application Security Testing) + DLP (Bloquea secretos antes del commit).
- `merci-hardening.py` : Inyección automática de políticas estrictas CSP (Content Security Policy).
- `merci-chaos.py` : Ingeniería del caos. Inyecta vulnerabilidades controladas para validar la resiliencia del sistema.

- `merci-linkcheck.py` : DAST (Dynamic Application Security Testing) que rastrea y bloquea despliegues si detecta enlaces rotos.

## IA & Orquestación

- `merci-brain.py` : Base de conocimiento estática (Shift-Left IA) para el asistente web.
- `merci-ssot.py` : Sincronización automática de documentación (Single Source of Truth).
- `merci-librarian.py` : Agente técnico que formatea cuadernos (Zero-Hallucination).
- `merci-blogger.py` : Agente de marketing para redacción DevRel.

## Publicación & Contenido

- `merci-publish.py` y `merci-wp.py` : Generador estático dual y publicador masivo Headless.
- `merci-linkedin.py` : Buffer asíncrono y motor de autenticación OIDC.
- `merci-optimizer.py` : Escalado y conversión a WebP automática.

## El Contrato de Desarrollo

---

La filosofía principal de Merci es el **Spec-Driven Development**. Todo clon del repositorio incluye un archivo `instrucciones.md`. No es una guía de uso, es un "contrato vinculante" que define exactamente qué está permitido hacer (ej. *Vanilla JS*) y qué está prohibido (ej. *Frameworks reactivos o estilos en línea*). El orquestador de *commits* valida estas especificaciones en cada guardado.

Si estás listo para adoptar una infraestructura donde el rendimiento y la seguridad no son una promesa, sino una métrica auditable, **eres bienvenido a clonarlo**. Para aquellos entornos donde el rendimiento y la seguridad no son una promesa, sino una métrica auditable, el repositorio base se encuentra disponible para su clonación y estudio.

 **Repositorio Oficial de Merci Boilerplate en GitHub** <https://github.com/MercedesDF/merci-boilerplate>